计算机学院实验报告

实验题目： liang-barsky 裁剪与中点划线法		学号：202200130077
日期：2024.4.16	班级：智能	姓名：薛润泽
Email：		

实验目的：

具体要求：
1，线段裁剪采用 Liang-baskey 算法
2，光栅化采用 Bresenham 算法，要求包括两点改进
3，在实验 2 作业框架的基础上，或其他框架的基础上；
4，算法结果需要可视化呈现在屏幕上；
5，支持键盘输入线段的任意起点和终点坐标；
6，实验周期为 1 周；

实验环境介绍：
Ubuntu

解决问题的主要思路：

1、liang-barsky 裁剪算法
2、中点画线法实现光栅化

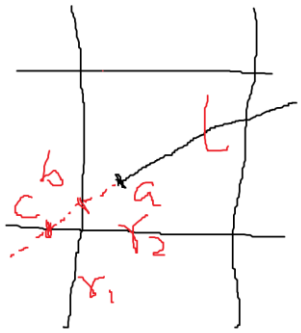
本实验的主要思路如下：

1. 学习 liang-barsky 裁剪算法，使用△x、△y、x1-xL，xR – x1、y1-xb，yt – y1 计算各个 r，并通过对 r 进行选择得到 u1，u2 从而得到裁剪线段。
2. 学习光栅图形学的中点划线法，通过将两个像素点的中心点带入直线求解，得到与 0 的大小关系，来选择像素点。

实验步骤与实验结果：

1. Liang-barsky 算法

Liang-barsky 算法的主要思想是“求交集”，以下图为例。



L 是实际线段，a 为 L 的入点，b 和 c 是 L 与两条入边的交点（上图 L 为示意图，举了 a 点在框内的情况，所以做了延长线交于 b，c 点）。△x、△y 分别是 x2-xa，y2-ya.也就是 L 线段的 x 轴上投影长度和 y 轴上投影长度。此时计算 xb-xa 与△x 的比值以及 yc-ya 与△y 的比值，然后再与 0 求最大值。实际上这是求了一个交集，即把 a 点限制在了框内。同理，对线段的另一个点也是这么做的。通过做两个交集，得到了裁剪线段的两个点，从而得到裁剪线段。

2、中点划线法

中点画线法主要目的是在两个相邻像素点中选择光栅化其中一个像素点，起到的作用是选择作用。通过计算相邻两个像素点的中点，将其代入到相应直线中，求整体方程的正负值，来判断直线更“接近”哪个像素点，从而选择更接近的像素点。

代码如下：

```
#include <bits/stdc++.h>
#include <chrono>
#include <iostream>
#include <opencv2/opencv.hpp>
#include <fstream>
#include <chrono>
using namespace cv;
const long long halfcube = 25;
const long long wholecube = 50;
const double startx = 100, starty = 100, endx = 600, endy =
600;
void paintline(cv::Mat & img, cv::Point &p1, cv::Point &p2){
    cv::circle(img, p1, 3, cv::Scalar(0, 0, 255), -1);
    cv::circle(img, p2, 3, cv::Scalar(0, 0, 255), -1);
    cv::line(img, p1, p2, cv::Scalar(0, 0, 255), 1);
}
```

```
void paintcube(cv::Mat & img, long long x, long long y){
    Rect rect = Rect(x, y, 50, 50);
    Scalar color = Scalar(0, 255, 0);
    rectangle(img, rect, color, 10, LINE_8);
}
void liangbasky(cv::Point &p1, cv::Point &p2){
    if(p1.x == p2.x){
        if(p1.y < 100) p1.y = 100;
        if(p2.y < 100) p2.y = 100;
        if(p1.y > 600) p1.y = 600;
        if(p2.y > 600) p2.y = 600;
        return ;
    }
    if(p1.y ==  p2.y){
        if(p1.x < 100) p1.x = 100;
        if(p2.x < 100) p2.x = 100;
        if(p1.x > 600) p1.x = 600;
        if(p2.x > 600) p2.x = 600;
        return;
    }
    if(p1.x > p2.x) swap(p1, p2);
    double inx = 100,  iny = 100 , outx = 600, outy = 600;
    if(p1.y > p2.y) swap(iny, outy);
    double deltax = p2.x - p1.x, deltay = p2.y - p1.y;
    double ul = (inx - p1.x) / deltax , ub = (iny - p1.y) /
deltay;
    double ut = std::abs((outx - p1.x ) / deltax), ur =
std::abs((outy - p1.y) / deltay);
    double res1 = std::max((double)0, std::max(ul, ub));
double res2 = std::min((double)1, std::min( ut, ur));
    p2.x = p1.x + res2 * deltax, p2.y = p1.y + res2 * deltay;
    p1.x += res1 * deltax, p1.y += res1 * deltay;
    return ;
}
void rasterization(cv::Mat &img, cv::Point p1, cv::Point p2){
    double A = -(p2.y - p1.y), B = p2.x - p1.x, C = p1.x *
p2.y - p2.x * p1.y;
    if(p1.x == p2.x){
        if(p1.y > p2.y) swap(p1.y, p2.y);
        double mvy = p1.y;
        long long tmpx = (long long)p1.x - (long long)p1.x %
50;
        long long tmpy = (long long)mvy - (long long)mvy % 50;
        while(abs(tmpy - p2.y) > 50){
```

```
            // if(abs(mvy - p2.y) <= 25) sym = 0;
            cv::Point paint1(tmpx, tmpy), paint2(tmpx+50,
tmpy+50);
            cv::rectangle(img, paint1, paint2, Scalar(255, 0,
0), 2, LINE_8);
            mvy += wholecube;
            tmpy = (long long)mvy - (long long)mvy % 50;
        }
            cv::Point paint1(tmpx, tmpy), paint2(tmpx+50,
tmpy+50);
            cv::rectangle(img, paint1, paint2, Scalar(255, 0,
0), 2, LINE_8);
        return ;
    }
    double k = -A / B;
    if(k >= 0 && k <= 1){
        if(p1.x > p2.x){
            std::swap(p1.x, p2.x); std::swap(p1.y, p2.y);
        }
        if(B < 0){
            A = -A; B = -B; C = -C;
        }
        std::cout<<" A " << A << " B "<<B<<" C "<<C<<'\n';
        int sym = 1;
        if(A * B < 0) sym = 1; // x up y up
        else sym = -1;// x up y down
        double mvx = p1.x, mvy = p1.y;
        long long tmpx = (long long)mvx - (long long)mvx % 50 +
25;// origin point
        long long tmpy = (long long)mvy - (long long)mvy % 50 +
25;
        int endsymbol = 1;
        while(endsymbol){
            std::cout<<endsymbol<<'\n';
            if(abs(tmpx - p2.x) <= 25) endsymbol = 0;
            // std::cout<<"count1"<<'\n';
            cv::Point paint1(tmpx - halfcube , tmpy - halfcube);
            cv::Point paint2(tmpx + halfcube, tmpy + halfcube);
            cv::rectangle(img, paint1, paint2, Scalar(255, 0,
0), 2, LINE_8);
            if(sym >0){
                if(A * (tmpx + wholecube ) + B * (tmpy + sym *
halfcube) + C > 0){
```

```
                std::cout<<" minor "<<A * (tmpx + wholecube )
+ B * (tmpy + sym * wholecube) + C <<'\n';
                tmpx += wholecube;
            }
            else{
                tmpx += wholecube;
                tmpy += sym * wholecube;
            }
        }
        else {
            if(A * (tmpx + wholecube ) + B * (tmpy + sym *
halfcube) + C < 0){
                tmpx += wholecube;
            }
            else{
                tmpx += wholecube;
                tmpy += sym * wholecube;
            }
        }
    }

   }
   else if(k >= -1 && k < 0){
       std::cout<<"k < 0 && k > -1"<<'\n';
       if(B < 0){
          A = -A; B = -B; C = -C;
       }
       int sym = 1;
       if(A * B < 0) sym = 1; // x up y up
       else sym = -1;// x up y down
       double mvx = p1.x, mvy = p1.y;
       long long tmpx, tmpy;
       tmpx = (long long)mvx - (long long)mvx % 50 + 25;//
origin point
       if((long long)mvy % 50 == 0) tmpy =  (long long)mvy -
(long long)mvy % 50 - 25;
       else tmpy = (long long)mvy - (long long)mvy % 50 + 25;
       int endsymbol = 1;
       while(endsymbol){
          std::cout<<endsymbol<<'\n';
          if(abs(tmpx - p2.x) <= 25) endsymbol = 0;
          // std::cout<<"count1"<<'\n';
          cv::Point paint1(tmpx - halfcube , tmpy - halfcube);
          cv::Point paint2(tmpx + halfcube, tmpy + halfcube);
```

```
          cv::rectangle(img, paint1, paint2, Scalar(255, 0,
0), 2, LINE_8);
          if(sym >0){
              if(A * (tmpx + wholecube ) + B * (tmpy + sym *
halfcube) + C > 0){
                  std::cout<<" minor "<<A * (tmpx + wholecube )
+ B * (tmpy + sym * wholecube) + C <<'\n';
                  tmpx += wholecube;
              }
              else{
                  tmpx += wholecube;
                  tmpy += sym * wholecube;
              }
          }
          else {
              if(A * (tmpx + wholecube ) + B * (tmpy + sym *
halfcube) + C < 0){
                  tmpx += wholecube;
              }
              else{
                  tmpx += wholecube;
                  tmpy += sym * wholecube;
              }
          }
       }
   }
   else if(k > 1){
       std::cout<<"k > 1"<<'\n';
       std::swap(p1.x, p1.y); std::swap(p2.x, p2.y);
std::swap(A, B);
       if(B < 0){
          A = -A; B = -B; C = -C;
       }
       int sym = 1;
       if(A * B < 0) sym = 1; // x up y up
       else sym = -1;// x up y down
       double mvx = p1.x, mvy = p1.y;
       long long tmpx = (long long)mvx - (long long)mvx % 50 +
25;// origin point
       long long tmpy = (long long)mvy - (long long)mvy % 50 +
25;
       int endsymbol = 1;
       while(endsymbol){
          if(abs(tmpx - p2.x) <= 25) endsymbol = 0;
```

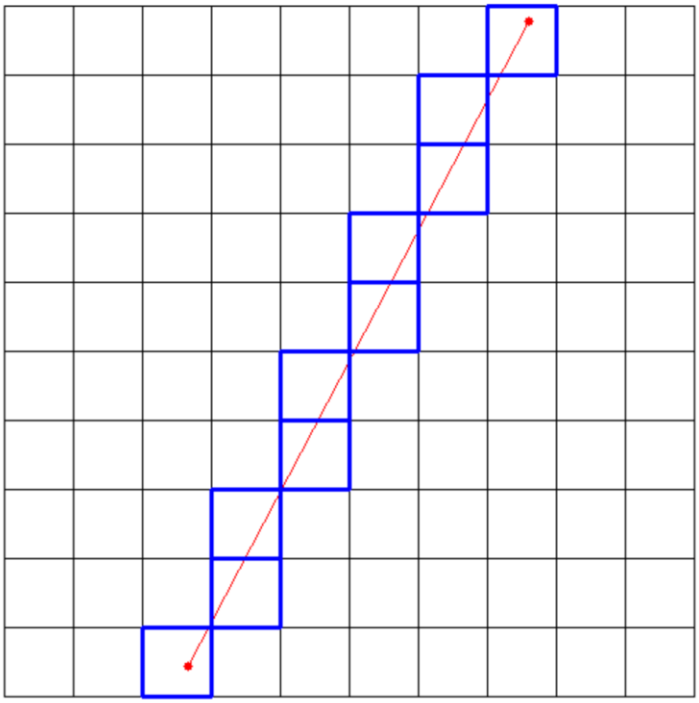
```
            // std::cout<<"count1"<<'\n';
            cv::Point paint1(tmpy - halfcube , tmpx - halfcube);
            cv::Point paint2(tmpy + halfcube, tmpx + halfcube);
            cv::rectangle(img, paint1, paint2, Scalar(255, 0,
0), 2, LINE_8);
            if(sym >0){
                if(A * (tmpx + wholecube ) + B * (tmpy + sym *
halfcube) + C > 0){
                    std::cout<<" minor "<<A * (tmpx + wholecube )
+ B * (tmpy + sym * wholecube) + C <<'\n';
                    tmpx += wholecube;
                }
                else{
                    tmpx += wholecube;
                    tmpy += sym * wholecube;
                }
            }
            else {
                if(A * (tmpx + wholecube ) + B * (tmpy + sym *
halfcube) + C < 0){
                    tmpx += wholecube;
                }
                else{
                    tmpx += wholecube;
                    tmpy += sym * wholecube;
                }
            }
        }
    }
    else if(k < -1){
        if(p1.y > p2.y){
            swap(p1.x, p2.x); swap(p1.y, p2.y);
        }
        double A = -(p2.y - p1.y), B = p2.x - p1.x, C = p1.x *
p2.y - p2.x * p1.y;
        if(A < 0){
            A = -A; B = -B; C = -C;
        }
        double mvx = p1.x, mvy = p1.y;
        long long tmpx, tmpy;
        tmpy = (long long)mvy - (long long)mvy % 50 + 25;//
origin point
        if(tmpx % 50 == 0)(long long)mvx - (long long)mvx % 50
- 25;
```

```
        else tmpx = (long long)mvx - (long long)mvx % 50 + 25;
        int endsymbol = 1;
        while(endsymbol){
            if(abs(tmpy - p2.y) <= 25) endsymbol = 0;
            // std::cout<<"count1"<<'\n';
            cv::Point paint1(tmpx - halfcube , tmpy - halfcube);
            cv::Point paint2(tmpx + halfcube, tmpy + halfcube);
            cv::rectangle(img, paint1, paint2, Scalar(255, 0,
0), 2, LINE_8);
                if(A * (tmpx - halfcube ) + B * (tmpy +
wholecube) + C < 0){
                    tmpy += wholecube;
                }
                else{
                    tmpy += wholecube;
                    tmpx -= wholecube;
                }
            std::cout<<abs(tmpy - p2.y)<<'\n';
                // std::cout<<A * (tmpx - halfcube ) + B * (tmpy
+ wholecube) + C<<'\n';
                // std::cout<<" tmpx "<< tmpx << " tmpy
"<<tmpy<<'\n';
            // if(counttime == 4)  break; counttime++;
        }
    }
}
int main(){
    cv::Mat img = Mat::zeros(Size(800, 800), CV_8UC3);
    img.setTo(255);             // 设置屏幕为白色
    for(int i = startx; i<=endx; i += 50){
        cv::Point p1(i, starty); cv::Point p2(i, endy);
        cv::Point p3(startx, i); cv::Point p4(endx, i);
        cv::line(img, p1, p2, cv::Scalar(0, 0, 0), 1);
        cv::line(img, p3, p4, cv::Scalar(0, 0, 0), 1);
    }
    double x1, y1, x2, y2;
    scanf("%lf%lf%lf%lf", &x1, &y1, &x2, &y2);
    cv::Point p1(x1, y1);
    cv::Point p2(x2, y2);
    liangbasky(p1, p2);

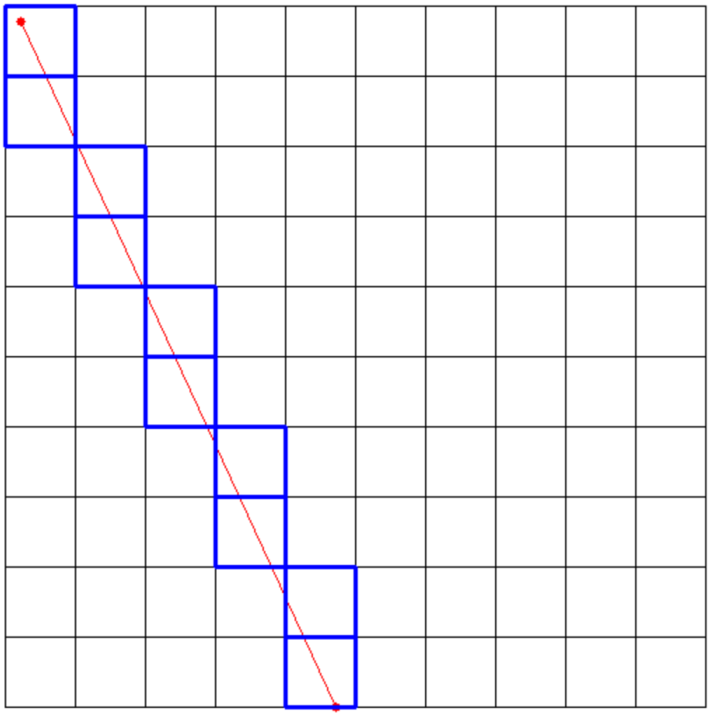
    paintline(img, p1, p2);
    cv::imshow("画板", img);
    rasterization(img, p1, p2);
```

```
    cv::imshow("画板", img);
    waitKey(0);
}
```

实验效果：
画线格的范围为（100,100）到（600，600），第一个截图画的是不需要裁剪的线段，第二个截图画的是需要裁剪的线段。
画线段（233,578），（480, 111）；



画线（111,111），（470,888），此线段需要裁剪。



实验中存在的问题及解决：

问题 1：直线的绘制需要讨论斜率是否大于 1
解决：将斜率大于 1 与小于 1 分开，大于 1 使用 y 做自变量，小于 1 使用 x 做自变量。

问题 2：直线的绘制需要讨论 x 增大时，y 是增大还是减小
解决：分类讨论，比如（100,100）到（200,150），则 x 增大时，y 增大，（100 400）（300,300），x 增大时，y 减小。